

1 Introduction

This report outlines the development workflow that was taken for each part of the coursework and justifies the design choices made, it then evaluates the results obtained and links it to how the design choices impacted the trade-off between speed and accuracy. The report is split into four sections, section three describes the deployment of an ANN locally and compares how compilation flags impact inference and memory usage. Section four compares the difference between local and remote deployment and how clock frequency has an impact on model latency. Section five covers the development, training and deployment of the sine wave model with results comparing the performance of the model using different runtime libraries. Section six discusses how the accuracy of the CNN based classification model can be improved by the modification of hyperparameters, it provides various tables comparing different epoch counts and improves the single test infer.py script to provide an inference on all the images. The results obtained indicated how the two-class model was limited and the why inclusivity and diversity should be considered when designing datasets.

Development was done using STM32CubeIDE V1.19.0. Part 1 and 3 were successfully tested on STM32CubeIDE V1.13.2, showing compatibility of code across different versions. The provided ReadMe files provide more insight on how to successfully run each part. The memory utilisation data was obtained using the build analyser section within STM32CubeIDE.

2 Contents

1	INTRODUCTION	1
2	CONTENTS	1
3	DEPLOYMENT OF AN ANN ONTO AN EMBEDDED SYSTEM.....	2
3.1	IMPLEMENTATION	2
3.2	EXPERIMENTAL RESULTS	2
4	DEPLOYMENT OF ML ALGORITHMS REMOTELY AND LOCALLY	3
5	DEVELOPMENT AND TRAINING OF A SINE-WAVE MODEL	4
5.1	CASE 1	4
5.2	CASE 1 RESULTS.....	5
5.3	CASE 2	5
5.4	CASE 2 RESULTS.....	6
5.5	CASE 3	6
5.6	CASE 3 RESULTS.....	7
6	DEPLOYMENT OF A CNN BASED FACE CLASSIFICATION MODEL.....	7
6.1	TASK 1	7
6.2	TASK 2	8
6.3	TASK 3	9
6.4	REFLECTION ON LIMITATIONS, DATASET DESIGN AND INCLUSIVITY	10
7	REFERENCES	10
8	APPENDIX.....	11

3 Deployment of an ANN onto an Embedded System

3.1 Implementation

The process of deploying the ANN onto the NUCLEO-F767ZI was done with the assistance of steps found in lab 2. The function prototypes were placed in the “USER CODE BEGIN PFP” section then the function definitions were placed in “USER CODE BEGIN 0”. The main function consisted of the variables which were assigned default values, the neural network instantiation, the test data for the neural network and a for loop that called the `neural_net_run` function, which looped through the test data.

```
1 start_time = getCurrentMicros();
2 result = neural_net_run(neural_net, test_data[i] + 1, 4);
3 end_time = getCurrentMicros();
```

Figure 1: Code snapshot for how the `getCurrentMicros` function was used.

To obtain the execution time the function `getCurrentMicros` was used. This function uses `systick` to measure the inference time. This was implemented so that it started and ended between the `neural_net_run` function, as shown in Figure 1.

This was considered as the correct timing method as the `printf` statement, which would output the results of the neural network would increase the execution time and this would not be considered the inference time of the model.

```
1 int i;
2 double *result;
3 int run;
4 int max_run = 10; // Change this value to change the number of runs.
5 uint32_t run_times[max_run];
6 uint32_t start_time;
7 uint32_t end_time;
8 uint32_t run_time;
9 uint32_t total_time = 0;
10 uint32_t avg_time = 0;
11 uint32_t mean = avg_time;
```

Figure 2: Code snapshot showing the variables used to obtain the average execution time.

As the inference time can change between each run due to differences in the pipeline of the processor, an average must be taken to make a fair judgement on the execution time. This was implemented using a second for loop. This was because the number of iterations is fixed and can be adjusted prior to runtime using the variable `max_run`, this value should match the length of the array `run_times` so that the values of each run could be stored, as shown in Figure 2. The `run_time` is calculated by the `end_time - start_time`. This is stored in the array and added to the `total_time`. This is repeated until the for loop reaches the `max_run`. Then the time of each run is output to the PuTTY terminal alongside the average inference time which is the `total_time / max_run`.

3.2 Experimental Results

The following results were obtained using STM32CubeIDE V1.19. The average execution time was obtained using a `max_run` set to 10.

Table 1 indicated that the O3 compilation flag was able to reduce the inference time of the model by about 49.79%. This is a benefit as the model now has a lower energy usage. The optimisation did not have a significant effect on the memory utilisation; this is understandable as loop unrolling takes place to reduce inference time and this process can sometimes result in changes to memory utilisation.

An advantage of memory usage not decreasing due to memory-level optimisation is that the model accuracy is not impacted. Techniques such as pruning aim to reduce memory usage can be an issue for neural networks as it can impact the model’s performance. Therefore, systems that require high levels of accuracy whilst still being able to run on resource constrained devices, such as those used in safety critical applications, should aim to reduce memory usage without this method.

25WSC334 - Deployment of Machine Learning Inference Models in IoT Devices F330590

Table 1: Results for execution time and memory utilisation of the ANN.

Model	Compilation Flag	Av. Inference Time (μ s)	Memory Utilisation (KB)	
			RAM	Flash
ANN	O0	19196	3.80	40.06
	O3	10362	3.80	35.54

4 Deployment of ML Algorithms Remotely and Locally

Table 2: Results for the deployment of the ResNet V1 image classification model on a remote and local board.

Model	Processor	Optimisation	Inference Time (μ s)	Memory Utilisation (KB)		Accuracy (%)
				RAM	Flash	
ST Edge AI Model Zoo	M33 (STM32H573i-DK @ 250 MHz)	N/A	85310	51.01	134.48	87.01
		Time	79940	64.18	134.45	87.01
	M7	N/A	52884	67.88	242.28	N/A
		Time	52884	67.88	242.28	N/A

Table 3: Results for the deployment of the BNN model on a remote and local board.

Model	Processor	Optimisation	Inference Time (μ s)	Memory Utilisation (KB)		Accuracy (%)
				RAM	Flash	
BNN on Cifar-10	M33 (STM32H573i-DK @ 250 MHz)	N/A	314900	149.97	556.89	100
		Time	314900	149.97	556.89	100
	M7	N/A	255818	151.77	725.84	N/A
		Time	255895	151.77	722.23	N/A

For remote deployment of both models the tflite file for the ResNet V1 and .h5 file for the BNN were uploaded to the ST developer cloud. Table 2 indicated that the developer cloud was successfully able to reduce the inference time, however for the BNN the optimisation did not impact the inference time. This could be because the BNN model was already well optimised.

5 Development and Training of a Sine-Wave Model

5.1 Case 1

The development and experimental results for the sine wave model were obtained using the following experimental setup; PyCharm 2025.2.4, Python 3.12, AMD Ryzen 5 5600X.

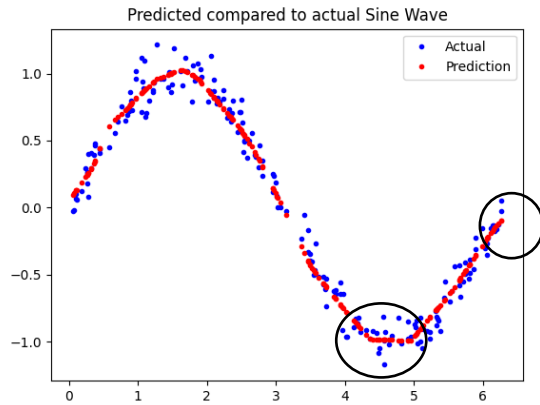


Figure 3: Sine wave prediction model with 865 total trainable parameters, x axis is the pi in radians, and the y axis is the amplitude.

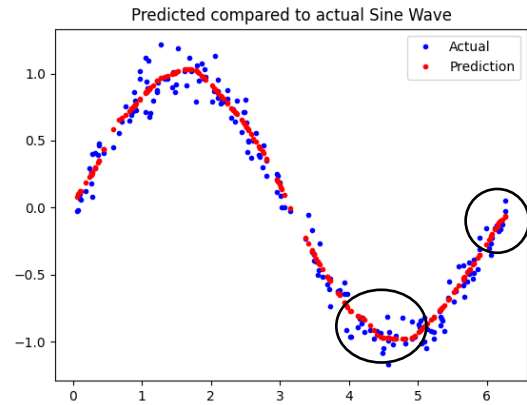


Figure 4: Sine wave prediction model (V2) with 3265 total trainable parameters, x axis is the pi in radians, and the y axis is the amplitude.

Using [1] the foundation of a sine wave model was developed. The original model was a three layer fully connected dense network. This meant for the project constraints the number of layers had to be increased by two.

The model was trained using 1000 randomly generated values between 0 and 2π . 600 values were used to train the model, 200 for validation and 200 for testing the model. The values within each split were also shuffled this ensured fair training of the model, rather than placing all training data in the first section of the sine wave. The five-layer dense neural network output in Figure 3 indicated that the sine wave prediction could still be improved further.

However, the biggest improvement in performance can be achieved by increasing the number of neurons in each layer, this will increase the number of total trainable parameters. The total trainable parameters were increased to 3265. The model architecture is shown in the appendix, this graph was obtained using the netron application. The optimiser was changed from rmsprop to adam. Previous research [2] indicated that the adam optimiser performed the best when compared to others. The combination of both these led to an improvement in the performance of the model as shown by Figure 4.

Increasing the number of trainable parameters also increased the time it takes for the model to train and will impact the inference time of the model. As such, this is a trade-off that needs to be carefully considered. Case 2 discusses the impact on inference time. Furthermore, by increasing the total trainable parameters the likelihood that the model overfits to the training dataset also increases. This is something that can be avoided by using a validation set, this ensures the training and validation loss converge to a certain value.

To deploy the model that was created using python onto the STM32CubeIDE, the model was converted into a C byte array, which was then used as a header file and implemented in main.c following the guidance in [3].

The mean absolute error was chosen as the primary evaluation metric as this represents the accuracy of the model better than the percentage derived for section four and six. Knowing the error between each prediction allows easier visualisation of where the model might be performing poorly. The average

25WSC334 - Deployment of Machine Learning Inference Models in IoT Devices F330590

inference time was obtained using the `getCurrentMicros` function for the same reason stated in section one.

5.2 Case 1 Results

Table 4: Results for the sine wave deployment using the TFLM runtime library.

Deployment	Processor	Optimisation	Av. Inference Time (μ s)	Memory Utilisation (KB)		Evaluation Metrics		
				RAM	Flash	MAE	MSE	RMSE
Case 1: CubeIDE	M7	N/A	234	20.48	107.59	0.0562	0.0043	0.0659

The results in Table 4 indicated that this deployment had the slowest inference time and the highest memory utilisation. The memory utilisation could be due to the large C byte array that the model is stored as. The lack of optimisations provided by the runtime libraries that were used in case 2 and 3, could explain why the inference time was so high. The error between the predicted value and actual value remained consistent with the results observed in Case 3. One possible explanation relates to where the inference code resides. For case 1 and 3 this inference code can be found in `main.c`, however the code for case 2 can be found in `app_x-cube-ai.c`.

5.3 Case 2

The model proposed in case 1 was converted into the tflite file format. The instructions provided in Lab 5 were used to assist with the deployment of the sine-wave model onto the NUCLEO-F767ZI using x-cube-ai.

```

COM7 - PUTTY
Test case 1: Input = 3.7699 rad (214.00 deg)
Output = [-0.977461 -0.567795 -0.033061] > Negative: Red LED ON
Inference time: 87 us

Test case 11: Input = 4.1033 rad (234.00 deg)
Output = [-0.923513 -0.566023 -0.497731] > Negative: Red LED ON
Inference time: 87 us

Test case 12: Input = 4.6077 rad (264.00 deg)
Output = [-0.876307 -0.894322 -0.016211] > Negative: Red LED ON
Inference time: 87 us

Test case 13: Input = 5.0245 rad (288.00 deg)
Output = [-0.820071 -0.951056 -0.030985] > Negative: Red LED ON
Inference time: 87 us

Test case 14: Input = 5.4664 rad (312.00 deg)
Output = [-0.685323 -0.743145 -0.057622] > Negative: Red LED ON
Inference time: 87 us

Test case 15: Input = 5.9643 rad (336.00 deg)
Output = [-0.276995 -0.406756 -0.029742] > Negative: Red LED ON
Inference time: 87 us

Test case 16: Input = 6.2032 rad (360.00 deg)
Output = [-0.000779 0.000000 0.000779] > Intersecting Zero: Both LEDs OFF
Inference time: 87 us

Average time : 89 us
MAE (Mean Absolute Error): 0.020584
MSE (Mean Squared Error): 0.001136
RMSE (Root Mean Squared Error): 0.033711

```

Figure 5: Structure of the terminal for the expected output.

The tflite file was uploaded as a network and the settings for the IOC file resembled the changes that were made in Lab 5. The `main.c` file calls `MX_X_CUBE_AI_Process()` from `../X-CUBE-AI/App/app_x-cube-ai.c`, to provide test inputs to the model. Figure 5 shows the predicted value, the actual value and the error between the two. The LEDs were used as a method of debugging and as a visual aid while the model was running.

As discussed in case 1, the change in location for the inference code might have had an impact on the results of the model.

5.4 Case 2 Results

Table 5: Results comparing two different implementations of the sine wave model using the CubeAI runtime library.

Deployment	Processor	Compiler Flag	Av. Inference Time (μ s)	Memory Utilisation (KB)		Evaluation Metrics		
				RAM	Flash	MAE	MSE	RMSE
V1: CubeAI [16,32,16,16,1]	M7	O3	61	8.08	43.91	0.028	0.0011	0.0349
V2: CubeAI [48,32,32,16,1]	M7	O3	89	8.14	48.16	0.0266	0.0012	0.0347

Two different models were deployed onto the board using CubeAI to investigate the impact of changing the number of trainable parameters. V1 was a sine wave model that had 1393 trainable parameters and V2 had 3265. Table 5 indicated that increasing the trainable parameters increased RAM size marginally with flash usage increasing more noticeably. This could be the result of the increased number of neurons which would impact the number of weights and biases, which are stored in flash as they only need to be read and not edited. There is also an increase in inference time of 20 μ s between V1 and V2, this can be considered a trade-off as the model accuracy did improve as shown by the reduction in error between the predicted value and actual value. As there were no design constraints, the choice was made to prioritise the accuracy of the model over inference time.

5.5 Case 3

For this section the tflite file that was created in case 1 and used in case 2 is uploaded to ST developer cloud. Initially ST Edge AI Core 2.1.0 was used. However, this version contained the x-cube-ai folder rather than the required AI folder. Therefore, ST Edge AI Core 4.0.0 version was used to obtain the required project file tree.

History of benchmark results

Show performance summary

Export as CSV

Refresh

Delete

☒	Date ↓	Model Name	Model Type	Version	Optimization	Use Neural-Art Accelerator™	Platform	MACC	Duration (ms)	Flash size	RAM size	Dev
☒	5/2/2026, 6:45:15 PM	sine_model.tflite	tflite	4.0.0-20500359356bb0	time	false	STM32 MCU	3393	0.05029	14.82 KiB	320 B	NU H7.
☒	5/2/2026, 6:44:31 PM	sine_model.tflite	tflite	4.0.0-20500359356bb0	ram	false	STM32 MCU	3393	0.05051	14.82 KiB	320 B	NU H7.
☒	5/2/2026, 6:42:31 PM	sine_model.tflite	tflite	4.0.0-20500359356bb0	balanced	false	STM32 MCU	3393	0.05032	14.82 KiB	320 B	NU H7.

Figure 6: Comparison of inference time using different optimisation techniques.

As shown in Figure 6, the optimisation did not change the flash or RAM usage, therefore the time optimised file was chosen, as this had the lowest inference time.

Once downloading the project file tree for the time optimised model, the main.c file was modified to get the desired output in the PuTTY terminal. The code closely resembles the code written for case 2. This was done so the output stays coherent with all cases in this section.

5.6 Case 3 Results

Table 6: Results for sine wave deployment using ST Edge AI runtime library.

Deployment	Processor	Optimisation	Av. Inference Time (μ s)	Memory Utilisation (KB)		Evaluation Metrics		
				RAM	Flash			
						MAE	MSE	RMSE
Case 3: ST Edge AI	M7	Time	84	12.09	55.08	0.0562	0.0043	0.0659

Table 6 indicated that deploying the model using the ST Edge AI runtime library did reduce the inference time of the model by 5μ s but at the cost of increasing memory utilisation. However, due to the change in runtime library, the error of the predicted values increased. This could have been caused by the difference in how floating-point operations are handled which could lead to the reduced precision but with the upside of a lower inference time.

Furthermore, the increase in error could suggest that the increase in trainable parameters might not have been a good addition to the model. Therefore, to validate this a lightweight model like V1 could be implemented using ST Edge AI to see if the error remains similar across all cases. This increase in error can be considered as a trade-off as it did successfully decrease inference time.

6 Deployment of a CNN Based Face Classification Model

6.1 Task 1

This section outlines the process taken to establish the highest possible training, validation and test accuracy for the model which could only predict two classes.

Table 7: Summary of accuracy across different number of epochs for the two-class model.

Accuracy	Accuracy across different number of epochs		
	8	16	32
Training	0.504	0.871	0.986
Validation	0.750	0.950	1.000
Test	0.717	0.817	0.983

25WSC334 - Deployment of Machine Learning Inference Models in IoT Devices F330590

```
(.venv) PS E:\STM32Projects\CW_Part_4> python infer.py --image data/test/circlepolis/circlepolis_190.png
Prediction: circlepolis
Confidence: 0.574
Warning: low confidence :-(. The image may belong to a face type that was not represented in training.
circlepolis 0.574
rectanglepolis 0.426

Training scope:
trained classes : ['circlepolis', 'rectanglepolis']
```

Figure 7: Snapshot of the output terminal showing a low confidence prediction on a model trained with 8 epochs.

The number of epochs in the original model was set to 8 epochs. The accuracy measured for 8 epochs could be considered low. Figure 7 indicated that when a circle was used as a test image for the infer.py script, the model made a low confidence but correct prediction. This suggested that the original model might not have had enough training time and was underfitting. To solve this, the number of epochs increased to 16 and 32.

Table 7 indicated that the accuracy of the model increased as the number of epochs was increased. However, it might not be useful to run the model for longer than 32 epochs, this is because for two classes the accuracy achieved is close enough to perfect and adding any more epochs would result in minimal gains when compared to 8 epochs and the amount of extra time the model would take to train.

An argument can also be made about only using 16 epochs to train the model as this would save more training time and therefore the choice between 16 and 32 epochs is dependent on the use case of the model. For example, a time sensitive problem such as a CCTV camera detecting a face might be more concerned with the speed of detection over the accuracy.

6.2 Task 2

This section outlines the misclassification of the unseen classes and the development of the inference script to run an inference on all the test data rather than on a single test image. It also evaluates the performance of the two-class model.

Table 8: Accuracy of predictions for inference across all images.

Model Type	Classes	Average accuracy	
		Number of epochs	
		32	64
Two Class	Circlepolis	0.900	0.933
	Rectanglepolis	0.986	0.900
Four Class	Circlepolis	1.000	0.933
	Ovalopolis	N/A	N/A
	Rectanglepolis	0.967	0.900
	Trianglepolis	N/A	N/A

The inference script was changed to iterate through every file located in the data/test directory using “python infer.py --test_dir data/test”. The option to inference a single image was maintained so that assessment of single images could still occur, this was helpful for understanding why some misclassifications occurred.

25WSC334 - Deployment of Machine Learning Inference Models in IoT Devices F330590

Table 9: Accuracy of prediction on a single image for each class and whether it was misclassified, using the two-class model.

Test image	Predicted class	Accuracy	Was the correct prediction made?
circlepolis_190	Circlepolis	0.999	Y
ovalopolis_185	Circlepolis	0.986	N
rectanglepolis_176	Rectanglepolis	0.997	Y
trianglepolis_170	Rectanglepolis	0.870	N

Table 9 indicated that the model, when trained using two classes with the number of epochs set to 8, misclassified the ovals and triangles. However, the drop in accuracy indicated that the model was slightly less confident that the image did belong in the predicted class. This showed that the training dataset was still useful for learning. However, it could be further improved by extending the training dataset to include images of the oval and triangle classes.

6.3 Task 3

```

1 # Original code for two classes
2 default=['circlepolis', 'rectanglepolis'],
3 # Modified code for four classes
4 default=['circlepolis', 'ovalopolis', 'rectanglepolis', 'trianglepolis'],

```

Figure 8: Code snapshot showing the modified list to include four classes.

Table 10: Comparison of accuracy across different number of epochs.

Classes	Average accuracy			
	Number of epochs			
	8	32	64	100
Circlepolis	0.800	0.633	0.667	0.467
Ovalopolis	0.567	0.867	0.900	0.900
Rectanglepolis	0.367	1.000	1.000	1.000
Trianglepolis	1.000	1.000	1.000	1.000
Average	0.683	0.875	0.892	0.842

The model was then retrained using 32 epochs and a new best.pt file was saved to the runs folder. The inference script was used to obtain the accuracy of the model. Table 10 indicated that although all four classes were classifying with a reasonably high accuracy, the circle and oval were still being misclassified more often than the other two. The number of epochs was increased to 64 and 100. Table 10 indicated that the increase in the number of epochs directly improved the accuracy of the prediction and reduced the misclassification of oval, however it seemed like there was an optimal value of 64 as when 100 epochs were tested the accuracy did slightly decrease, this could be a sign of overfitting. This shows the importance of not overtraining a ML model. Approaches such as early stopping can be used to prevent overfitting as once the optimal epoch is found the model does not conduct any more runs through the training data.

6.4 Reflection on Limitations, Dataset Design and Inclusivity

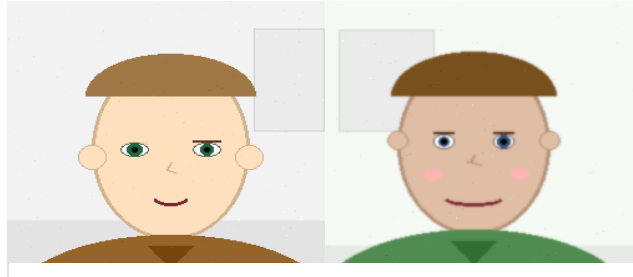


Figure 9: Comparing the similarity between Circleopolis_171.png and ovalopolis_187.png.

The quality of the dataset can also have an impact on the accuracy of the model. As shown in Figure 9, the two images are quite similar, this showed that the choice of dataset could have a direct influence on the accuracy of predictions. In the case of this model, the choice of dataset resulted in misclassification of some circles as ovals. This therefore could also explain why the circle class had the lowest accuracy when compared to the other three classes.

Task 2 indicated that providing limited training data for the model to learn from can cause misclassification. This should be avoided in the real world as a model like this could be deployed onto a camera and used to detect weapons. If the models training dataset did not contain a certain weapon, it would fail to identify it and be considered as a major flaw of the system. Alternatively, if it was used in the medical imaging sector to detect tumours, a model without sufficient training data might misclassify a tumour or fail to identify one. Both errors could lead to direct consequences to the patient and result in financial or legal problems for the institution that is operating the machine.

In real-world systems, excluding demographic groups from training data creates algorithmic bias. This can result in systematically worse performance for underrepresented groups, leading to a form of automated discrimination with consequences on fairness and equality.

7 References

- [1] "Intro to TinyML Part 1: Training a Model for Arduino in TensorFlow," *DigiKey*, 2020. <https://www.digikey.com/en/maker/projects/intro-to-tinyml-part-1-training-a-model-for-arduino-in-tensorflow/8f1fc8c0b83d417ab521c48864d2a8ec>
- [2] A. Zohrevand and Z. Imani, "An Empirical Study of the Performance of Different Optimizers in the Deep Neural Networks," *2022 International Conference on Machine Vision and Image Processing (MVIP)*, Feb. 2022, doi: <https://doi.org/10.1109/MVIP53647.2022.9738743>.
- [3] "TinyML: Getting Started with TensorFlow Lite for Microcontrollers," *DigiKey*, 2020. <https://www.digikey.com/en/maker/projects/tinyml-getting-started-with-tensorflow-lite-for-microcontrollers/c0cdd850f5004b098d263400aa294023> (accessed May 05, 2026).

8 Appendix

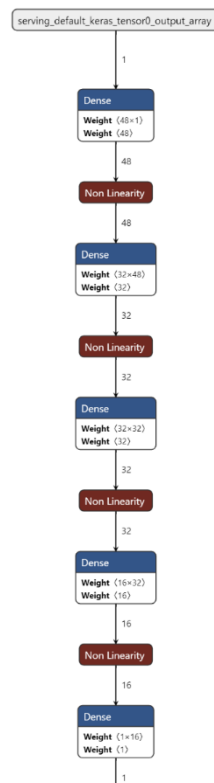


Figure 10: Five-layer
dense neural network
architecture.

Table 11: Summary of results for all forms of deployment for section 5.

Deployment	Processor	Optimisation	Av. Inference Time (µs)	Memory Utilisation (KB)		Evaluation Metrics		
				RAM	Flash	MAE	MSE	RMSE
Case 1: CubeIDE	M7	N/A	234	20.48	107.59	0.0562	0.0043	0.0659
Case 2: CubeAI	M7	N/A	89	8.14	48.16	0.0266	0.0012	0.0347
Case 3: ST Edge AI	M7	Time	85	12.04	55.08	0.0562	0.0043	0.0659